



Cardmarket Yu-Gi-Oh! Card List Scraper - Complete Documentation

Table of Contents

1. [Overview](#)
2. [System Architecture](#)
3. [Prerequisites](#)
4. [Input/Output Files](#)
5. [Core Components](#)
6. [Configuration Parameters](#)
7. [Scraping Logic and Concepts](#)
8. [Empty Detection System](#)
9. [Card Data Extraction](#)
10. [Anti-Detection Mechanisms](#)
11. [Error Handling and Recovery](#)
12. [Progress Management](#)
13. [Usage Instructions](#)
14. [Troubleshooting](#)

Overview

The **Cardmarket Card List Scraper** is a Python-based web scraping tool designed to extract comprehensive Yu-Gi-Oh! card data from [Cardmarket.com](https://cardmarket.com). The script processes a list of expansions (card sets) and retrieves detailed information about individual cards that have active offers available for purchase.

Purpose

- Extract card listings from all Yu-Gi-Oh! expansions on Cardmarket
- Gather card metadata including card name, ID, number, rarity, and URL
- Identify and categorize empty expansions (no available singles)
- Handle rate limiting, errors, and edge cases gracefully
- Provide resumable execution with checkpoint system

Key Features

- **Two-phase execution:** Fast initial run followed by careful recovery
- **Parallel processing:** Multiple concurrent workers for efficiency
- **Intelligent empty detection:** Three-tier system for identifying empty expansions
- **Comprehensive error handling:** Retry logic with exponential backoff
- **Progress persistence:** Checkpoint system for resumable execution
- **Anti-detection measures:** Session pooling, rate limiting, and realistic headers

System Architecture

Two-Phase Approach

Phase 1: Fast Main Run

- **Purpose:** Process all expansions quickly with reasonable reliability
- **Workers:** 8 parallel workers
- **Speed:** 3 requests per second
- **Retries:** 2 attempts per expansion
- **Retry Delay:** 12-18 seconds (randomized)
- **Outcome:** Successful extractions, empty expansions, and rejected expansions

Phase 2: Careful Recovery

- **Purpose:** Retry rejected expansions with maximum care (empty expansions are NOT retried)
- **Workers:** 1 single-threaded worker
- **Speed:** 1 request per second
- **Retries:** 5 attempts per expansion
- **Retry Delay:** 20-30 seconds (randomized)
- **Session Warmup:** 10 seconds between fresh sessions
- **Outcome:** Recovered data, confirmed empty, or permanently rejected

Design Rationale

The two-phase approach balances speed and reliability:

1. **Phase 1** maximizes throughput by processing expansions in parallel
2. **Phase 2** focuses on quality by giving rejected expansions extra attention with fresh sessions and longer delays

3. Empty expansions are correctly identified and NOT retried (they're genuinely empty, not errors)

Prerequisites

Required Python Libraries

```
import json
import cloudscraper
from bs4 import BeautifulSoup
import time
import re
import random
from datetime import datetime
from concurrent.futures import ThreadPoolExecutor, as_completed
import threading
from pathlib import Path
```

Installation

```
pip install cloudscraper beautifulsoup4
```

System Requirements

- Python 3.7 or higher
- Stable internet connection
- Sufficient disk space for output files (~50-100 MB)

Input/Output Files

Input File

cardmarket_expansion_list.json

- Contains all Yu-Gi-Oh! expansion metadata
- Required structure:

```
[
  {
    "expansion_id": 1234,
    "expansion_name": "Example Set",
    "expansion_code": "EXMP"
  }
]
```

Output Files

cardmarket_card_list.json

- Primary output containing all extracted card data
- Structure:

```
[
  {
    "expansion_id": 1234,
    "expansion_name": "Example Set",
    "expansion_code": "EXMP",
    "card_id": 567890,
    "card_name": "Dark Magician",
    "card_number": "EXMP-EN001",
    "card_rarity": "Ultra Rare",
    "card_url": "https://www.cardmarket.com/en/YuGiOh/Products/Singles/..."
  }
]
```

cardmarket_empty_expansions.json

- Lists expansions with no available single cards
- Contains the same expansion metadata from input file

cardmarket_rejected_expansions.json

- Lists expansions that failed to scrape after all retry attempts
- Includes detailed error information:

```
[
  {
    "expansion_id": 1234,
    "expansion_name": "Example Set",
    "total_attempts": 5,
    "attempts_detail": [
      {
        "attempt": 1,
        "card_count": 0,
        "issues": ["Page 1: 403 Forbidden"],
        "html_errors": [...]
      }
    ],
    "timestamp": "2025-10-26T18:00:00",
    "phase": 2
  }
]
```

Checkpoint Files

`cardmarket_checkpoint.json`

- Tracks Phase 1 progress
- Allows resumption after interruption
- Deleted after Phase 1 completion

`cardmarket_recovery_checkpoint.json`

- Tracks Phase 2 progress
- Deleted after Phase 2 completion

Core Components

Classes

`RateLimiter`

Controls request frequency to avoid overwhelming the server.

Key Methods:

- `wait(jitter=0.3)`: Enforces minimum time between requests with optional random jitter

Purpose:

- Prevents rate limiting (HTTP 429 errors)
- Mimics human browsing patterns
- Configurable per phase (3 req/sec Phase 1, 1 req/sec Phase 2)

`SessionPool`

Manages multiple scraper sessions for parallel workers.

Key Methods:

- `get_session(worker_id)`: Returns existing or creates new session
- `refresh_session(worker_id)`: Forces creation of new session

Features:

- Session reuse (up to 10 requests per session)
- Per-worker session isolation
- Automatic session refresh after error

Purpose:

- Connection persistence (faster requests)

- Worker isolation (parallel processing)
- Session rotation (anti-detection)

Key Functions

Configuration Functions

- `create_fresh_scraper(worker_id)`: Creates new cloudscraper instance with realistic headers
- `warmup_session(scraper, delay)`: Warms up session by visiting homepage

Page Fetching

- `fetch_page(scraper, url, rate_limiter, retries)`: Fetches HTML with error handling
 - Returns: `(html, final_url, error, error_detail)`
 - Handles HTTP errors (403, 429, 503)
 - Implements retry logic with delays

Empty Detection (Three-Tier System)

1. `check_if_product_page_redirect(html)`: Detects redirect to product page
 - Looks for "Available items" text in `<dt>` tags
 - Product pages have this, search results don't
2. `check_if_empty_on_first_page(html)`: Detects explicit "no results" message
 - Checks for "Sorry, no matches for your query"
 - Only checked on page 1
3. `check_if_only_sealed_products(html)`: Detects sealed products without singles
 - Looks for `productRow` divs
 - Checks if any link to `/Products/Singles/`
 - If rows exist but no Singles links = only sealed products

Card Data Extraction

- `extract_card_number_comprehensive(row, card_name, row_text, parts)`: Multi-strategy card number extraction
 - Strategy 1: After card name in text
 - Strategy 2: HTML structure inspection
 - Strategy 3: Aggressive pattern matching
 - Strategy 4: After expansion code
- `extract_cards_from_html(html, expansion_id, expansion_name, expansion_code)`: Parses HTML for cards

- Finds all `productRow` divs
- Extracts card ID from row ID attribute
- Finds Singles links only
- Extracts card number and rarity
- Returns list of card dictionaries

Main Scraping Logic

- `scrape_expansion_with_retry(...)`: Core scraping logic for one expansion
 - Iterates through pages until no more results
 - Applies all empty detection checks
 - Handles pagination (0.4s delay between pages, 1-2s in recovery)
 - Returns: (cards, expansion_code, fetch_issues, html_errors, is_empty)
- `scrape_expansion_worker(...)`: Worker wrapper with retry logic
 - Executes up to `max_retries` attempts
 - Tracks all attempts with detailed logging
 - Determines final status: success, empty, or rejected
 - Returns comprehensive result dictionary

Phase Execution

- `phase1_fast_run(expansions, start_idx)`: Executes Phase 1
 - Creates session pool and rate limiter
 - Spawns worker threads
 - Displays real-time progress
 - Saves every 50 expansions
 - Returns cards, empty list, rejected list, stats
- `phase2_recovery(expansions_dict, rejected_list, all_cards)`: Executes Phase 2
 - Processes only rejected expansions (not empty)
 - Single-threaded with maximum care
 - Fresh session per expansion
 - Longer delays and more retries
 - Can reclassify rejected → empty

Persistence

- `save_data(all_cards, expansions, empty_expansions, rejected_expansions)`: Thread-safe file I/O
 - Uses file lock for concurrent access
 - Updates all output files atomically
 - JSON format with UTF-8 encoding

Configuration Parameters

Phase 1 Settings

```
PHASE1_MAX_WORKERS = 8           # Parallel workers
PHASE1_REQUESTS_PER_SECOND = 3   # Request rate
PHASE1_MAX_RETRIES = 2           # Retry attempts
PHASE1_RETRY_DELAY = (12, 18)    # Delay range in seconds
```

Phase 2 Settings

```
PHASE2_MAX_WORKERS = 1           # Single-threaded
PHASE2_REQUESTS_PER_SECOND = 1   # Slower rate
PHASE2_MAX_RETRIES = 5           # More retries
PHASE2_RETRY_DELAY = (20, 30)    # Longer delays
PHASE2_SESSION_WARMUP = 10       # Session warmup delay
```

Session Management

```
SESSION_REUSE_COUNT = 10         # Requests per session before refresh
SESSION_WARMUP_DELAY = 2         # Homepage visit delay
RANDOM_JITTER = 0.3              # Random delay variance
```

User Agents

Five rotating user agents mimicking Chrome, Firefox, and Edge on Windows/Mac:

```
USER_AGENTS = [
    'Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36...',
    'Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:121.0) Gecko...',
    ...
]
```


Scraping Logic and Concepts

URL Construction

```
url = (  
    f"{BASE_URL}/en/YuGiOh/Products/Search?"  
    f"searchMode=v1&idCategory=0&idExpansion={expansion_id}"  
    f"&onlyAvailable=on&idRarity=0&site={page}"  
)
```

Key Parameters:

- `searchMode=v1`: Use version 1 search
- `idCategory=0`: All categories (defaults to Singles when available)
- `idExpansion={id}`: Specific expansion ID
- `onlyAvailable=on`: **CRITICAL** - Only show cards with active offers
 - Prevents listing sold-out cards
 - Ensures data relevance for buyers
 - May cause pagination issues (why we handle redirects)
- `idRarity=0`: All rarities
- `site={page}`: Page number (1-indexed)

Why `&onlyAvailable=on` is Essential

Without this filter:

- Script would collect cards with NO available offers
- Data would be useless for purchasing
- Would include historical/discontinued items
- Output would be cluttered with unavailable cards

With this filter:

- Only cards actively for sale are included
- Data is immediately actionable
- Reflects current market availability
- May result in "empty" expansions if nothing is for sale

Pagination Logic

1. Start with page = 1
2. Fetch page and check for empty conditions
3. If cards found, extract them
4. If no cards found on page > 1, end pagination (natural end)
5. If no cards on page 1, mark as empty or rejected based on reason
6. Increment page and repeat with short delay

Important: Pages without results on page > 1 are normal (end of results), not errors.

Status Classification

Success ✓

- **Criteria:** At least 1 card extracted from Singles
- **Meaning:** Expansion has available single cards for sale
- **Example:** Standard booster sets with singles market

Empty ∅

- **Criteria:** One of three empty conditions met on page 1
- **Meaning:** Expansion genuinely has no available singles
- **Examples:**
 - Sealed product only (tins, structure decks)
 - Out of print with no market
 - Redirects to product page
 - "Sorry, no matches" message

Failed ✗

- **Criteria:** Errors prevent data extraction after all retries
- **Meaning:** Technical issue, not inherent to expansion
- **Examples:**
 - HTTP 403 Forbidden (IP blocked)
 - HTTP 429 Rate Limited
 - Connection timeout
 - Invalid HTML response

Empty Detection System

Three-Tier Detection Logic

Tier 1: Product Page Redirect

When it happens:

- Cardmarket redirects search to specific product page
- Usually for sealed-only expansions or single-item sets

Detection method:

```
def check_if_product_page_redirect(html):  
    # Look for "Available items" in <dt> tags  
    # Product pages have this, search results don't
```

Examples:

- Starter Deck → Single sealed product page
- Special promo tin → Product page for tin itself

Why it's empty:

- No searchable list of singles
- Only sealed product available
- Search function bypassed by redirect

Tier 2: Explicit Empty Message

When it happens:

- Page 1 displays "Sorry, no matches for your query"
- No product rows exist

Detection method:

```
def check_if_empty_on_first_page(html):  
    if 'Sorry, no matches for your query' in html:  
        return True
```

Only checked on page 1 because:

- Page 1 empty = genuinely no results
- Page 2+ empty = natural end of results (not an error)

Examples:

- Brand new expansion not yet released

- Out-of-print set with zero market
- Expansion with all cards delisted

Tier 3: Sealed Products Only

When it happens:

- Page has `productRow` elements
- But none link to `/Products/Singles/`
- Only links to `/Products/Collector-Tins/`, `/Products/Storage/`, etc.

Detection method:

```
def check_if_only_sealed_products(html):
    # Has rows but no Singles links
    product_rows = soup.find_all('div', id=re.compile(r'^productRow\d+'))
    if not product_rows:
        return False

    # Check for Singles links
    has_singles = any(
        row.find('a', href=re.compile(r'/en/YuGiOh/Products/Singles/'))
        for row in product_rows
    )

    return not has_singles # Empty if no singles
```

Example case:

- Expansion ID 3224 (2020 Tin of Lost Memories)
- Has 3 product rows:
 1. Empty Tin (storage product)
 2. Sealed Tin (collector product)
 3. 12x Sealed Tins (bulk product)
- Zero singles available
- Result: Marked as Empty ∅

Why it's empty:

- Expansion only sells sealed products
- No individual cards broken out for sale
- Singles market doesn't exist for this expansion

Card Data Extraction

Card Number Extraction

Card numbers are crucial for identification but inconsistently formatted in HTML. The script uses **four extraction strategies** in sequence:

Strategy 1: After Card Name

- Parses row text split by delimiters
- Finds card name, then looks for pattern after it
- Patterns: `[A-Z]{1,5}[0-9]{1,3}`, `EXMP-EN001`, etc.

Strategy 2: HTML Structure

- Inspects specific column classes
- Looks in `col-md-2` with `d-lg-flex` classes
- Direct text extraction from number column

Strategy 3: Aggressive Pattern Matching

- Splits by all whitespace and pipes
- Matches card number patterns anywhere in row
- Filters out prices (contains €, commas)
- Filters out large pure numbers (> 100)

Strategy 4: After Expansion Code

- Finds expansion code (2-5 capital letters)
- Looks for number immediately after
- Useful when name extraction fails

Why comprehensive extraction?

- Cardmarket HTML structure varies by expansion
- Some expansions have inconsistent layouts
- Old expansions use different HTML patterns
- Multi-strategy ensures maximum coverage

Card Rarity Extraction

Rarity is stored in SVG attributes with multiple possible locations:

Priority order:

1. `aria-label` attribute on SVG

2. `data-bs-original-title` attribute
3. `title` attribute
4. Any attribute containing rarity keywords (rare, common, super, ultra, secret)
5. Specific column classes with SVG children

Example rarities:

- Common
- Rare
- Super Rare
- Ultra Rare
- Secret Rare
- Starlight Rare

Card ID Extraction

```
# From productRow ID attribute
row_id = row.get('id', '') # e.g., "productRow567890"
card_id_match = re.search(r'productRow(\d+)', row_id)
card_id = int(card_id_match.group(1)) # 567890
```

Singles Link Validation

Critical filter:

```
card_link = row.find('a', href=re.compile(r'/en/YuGiOh/Products/Singles/'))
if not card_link:
    continue # Skip sealed products
```

This ensures:

- Only actual cards are extracted
- Sealed products are ignored
- Tins, boxes, and accessories are excluded
- URL structure validates card type

Anti-Detection Mechanisms

Session Management

Session Pooling:

- Each worker maintains its own session
- Sessions reused for up to 10 requests
- Reduces connection overhead
- Mimics persistent browser behavior

Session Refresh:

- After 10 requests, new session created
- On 403 errors, immediate refresh
- Fresh cookies and connection state

Session Warmup:

```
def warmup_session(scraper, delay=2):  
    # Visit homepage before scraping  
    scraper.get(f"{BASE_URL}/en/YuGiOh")  
    time.sleep(delay)
```

- Establishes legitimate browsing pattern
- Loads site cookies and JavaScript
- Appears more human-like to server

Realistic Headers

```
scraper.headers.update({  
    'User-Agent': <rotating user agent>,  
    'Accept': 'text/html,application/xhtml+xml...',  
    'Accept-Language': 'en-US,en;q=0.9,de;q=0.8',  
    'Accept-Encoding': 'gzip, deflate, br',  
    'DNT': '1',  
    'Connection': 'keep-alive',  
    'Upgrade-Insecure-Requests': '1',  
    'Sec-Fetch-Dest': 'document',  
    'Sec-Fetch-Mode': 'navigate',  
    'Sec-Fetch-Site': 'none',  
    'Sec-Fetch-User': '?1',  
    'Cache-Control': 'max-age=0',  
    'Referer': f"{BASE_URL}/en/YuGiOh"  
})
```

Key headers:

- User-Agent: Rotates between 5 realistic agents
- Accept-*: Mimics browser content negotiation

- `Sec-Fetch-*`: Modern browser security headers
- `Referer`: Shows navigation from site homepage
- DNT: Do Not Track flag (privacy-conscious)

Rate Limiting

Phase 1:

- 3 requests per second = 180 requests per minute
- Random jitter (0-0.3s) added to each request
- Mimics variable human click speed

Phase 2:

- 1 request per second = 60 requests per minute
- Extra cautious to avoid triggering rate limits
- Suitable for recovery after Phase 1 failures

Between pages:

- 0.4s delay in Phase 1 (fast pagination)
- 1-2s delay in Phase 2 (careful pagination)

CloudScraper

Uses `cloudscraper` library (not plain requests):

- Automatically solves JavaScript challenges
- Handles Cloudflare protection
- Executes anti-bot checks transparently
- More sophisticated than basic HTTP requests

Error Handling and Recovery

HTTP Error Responses

403 Forbidden

Meaning: IP temporarily blocked or access denied

Response:

- In Phase 1: Refresh session, wait 30-45s, retry once
- In Phase 2: Fresh session, wait longer, retry up to 5 times
- If persistent: Mark as rejected with error details

429 Too Many Requests

Meaning: Rate limit exceeded

Response:

- Wait $10 \times \text{attempt_number}$ seconds
- Retry automatically
- Does NOT count toward max retries
- Respects server's rate limiting

503 Service Unavailable

Meaning: Server temporarily down

Response:

- Wait 5 seconds
- Retry immediately
- Usually transient issue

Retry Logic

Phase 1 retries (2 attempts):

```
for attempt in range(1, 3): # 2 attempts total
    # Try to scrape
    if card_count > 0:
        break # Success
    if is_empty:
        break # Genuinely empty
    # Otherwise retry after delay
```

Phase 2 retries (5 attempts):

```
for attempt in range(1, 6): # 5 attempts total
    # Fresh session
    scraper = create_fresh_scraper(worker_id)
    warmup_session(scraper, 10) # Longer warmup

    # Try to scrape with more care
    # 20-30 second delays between retries
```

Error Tracking

Every failed attempt is logged with:

```
{
  'attempt': 1,
  'card_count': 0,
  'issues': ['Page 1: 403 Forbidden'],
  'html_errors': [
    {
      'page': 1,
      'url': '...',
      'error': '403 Forbidden',
      'detail': '403 Forbidden (attempt 1/3)',
      'html_snippet': '<html>...' # First 500 chars
    }
  ]
}
```

Saved to rejected expansions JSON for debugging.

Best Result Selection

If multiple attempts yield different results:

```
best_card_count = 0
for attempt in attempts:
    if attempt['card_count'] > best_card_count:
        best_result = attempt
        best_card_count = attempt['card_count']
```

Use the attempt that extracted the most cards (handles partial failures).

Progress Management

Checkpoint System

Phase 1 Checkpoint

File: cardmarket_checkpoint.json

Content:

```
{
  "last_expansion_idx": 1234
}
```

Behavior:

- Saved every 50 expansions
- Allows resumption from last checkpoint
- Deleted after Phase 1 completes
- Script resumes from `last_expansion_idx + 1`

Phase 2 Checkpoint

File: `cardmarket_recovery_checkpoint.json`

Content:

```
{
  "last_processed": 12
}
```

Behavior:

- Saves every 10 rejected expansions processed
- Tracks progress through rejected list
- Deleted after Phase 2 completes

Resumption Logic

```
# On startup
if Path(CHECKPOINT_FILE).exists():
    checkpoint = json.load(open(CHECKPOINT_FILE))
    start_idx = checkpoint['last_expansion_idx'] + 1
    print(f"Resuming from expansion #{start_idx}")
else:
    start_idx = 0  # Start from beginning
```

Interruption Handling

Ctrl+C (KeyboardInterrupt):

```
try:
    main()
except KeyboardInterrupt:
    print("\n△ Interrupted - progress saved")
```

- Gracefully saves current progress
- Next run resumes from last checkpoint
- No data loss

Incremental Saves

Data is saved:

- Every 50 expansions in Phase 1
- Every 10 expansions in Phase 2
- After each phase completes
- On script interruption (if in save window)

Benefits:

- Minimal data loss on crash
- Can inspect partial results
- Long-running scrapes are safe

Usage Instructions

Initial Setup

1. Ensure input file exists:

```
# cardmarket_expansion_list.json must be in same directory
ls cardmarket_expansion_list.json
```

2. Install dependencies:

```
pip install cloudscraper beautifulsoup4
```

3. Verify Python version:

```
python --version # Should be 3.7+
```

Running the Script

```
python cardmarket_card_list_scraper.py
```

Expected Console Output

```
=====
CARDMARKET SCRAPER v10.0 - Complete
=====

=====
Loading Expansions
=====
✓ Loaded 2422 expansions
```

```

=====
PHASE 1: FAST MAIN RUN
=====
Workers: 8 | Speed: 3 req/sec

[ 1/2422] 2-Player Starter Deck... (ID: 1651) | 40 cards | ✓ Success | 0.0%
[ 2/2422] 2-Player Starter Set (ID: 5420) | 0 cards | ∅ Empty | 0.1%
[ 3/2422] 2013 Zexal Collection Tin (ID: 1433) | 0 cards (R2) | ✗ Failed | 0.1%
...

=====
PHASE 1 COMPLETE
=====
Total cards: 128184
Successful: 2100
Empty: 250
Rejected: 72
Duration: 2:15:30
Speed: 1078 exp/hour

⚠ Phase 1 left 72 REJECTED expansions
  (Empty expansions are normal and won't be retried)
  Starting Phase 2 recovery for rejected only...

=====
PHASE 2: CAREFUL RECOVERY (Rejected Only)
=====
Workers: 1 | Speed: 1 req/sec
Retries: 5 | Warmup: 10s
Rejections to process: 72

[ 1/ 72] Battle Pack Epic Dawn (ID: 1371) | ✓ Recovered 32 cards!
[ 2/ 72] Ancient Sanctuary (ID: 1012) | ∅ Confirmed empty
[ 3/ 72] Some Other Set (ID: 5555) | ✗ Still rejected
...

=====
PHASE 2 COMPLETE
=====
Recovered: 45/72
Now confirmed empty: 20
Still rejected: 7
Duration: 0:25:15

=====
FINAL SUMMARY
=====
Total cards: 129640
Successful expansions: 2145
Empty expansions: 270
Phase 1 rejections: 72
Phase 2 recovered: 45
Final rejections: 7
Overall duration: 2:40:45
=====

```

✓ Complete!

Resuming After Interruption

If script is stopped (Ctrl+C or crash):

```
python cardmarket_card_list_scraper.py
```

Output will show:

```
✓ Resuming Phase 1 from expansion #1235
```

Script continues where it left off.

Output File Locations

After completion, check:

```
ls -lh cardmarket*.json
# -rw-r--r-- cardmarket_card_list.json      (45 MB)
# -rw-r--r-- cardmarket_empty_expansions.json (15 KB)
# -rw-r--r-- cardmarket_rejected_expansions.json (2 KB)
```

Troubleshooting

Common Issues

1. Rate Limited (429 errors)

Symptoms:

```
✗ Failed | HTTP 429: Too Many Requests
```

Solutions:

- Wait 1-2 hours before restarting
- Script will automatically retry with delays
- Reduce PHASE1_REQUESTS_PER_SECOND to 2
- Increase PHASE1_RETRY_DELAY to (20, 30)

2. IP Blocked (403 errors)

Symptoms:

X Failed | HTTP 403: Forbidden

Solutions:

- Change network (use mobile hotspot, VPN)
- Wait 24 hours for block to clear
- Phase 2 will retry with fresh sessions
- Contact Cardmarket if persistent

3. No Cards Extracted

Symptoms:

[50/2422] Some Set (ID: 1234) | 0 cards | ∅ Empty

Analysis:

- Check if expansion is genuinely empty
- Visit URL manually: <https://www.cardmarket.com/en/YuGiOh/Products/Search?searchMode=v1&idCategory=0&idExpansion=1234&onlyAvailable=on&idRarity=0>
- If cards visible but not extracted:
 - May be sealed products only (working as intended)
 - May be HTML structure change (needs script update)
 - Check `cardmarket_rejected_expansions.json` for error details

4. Script Stops/Freezes

Symptoms:

- No output for > 5 minutes
- Script appears hung

Solutions:

- Press Ctrl+C to interrupt gracefully
- Check network connection
- Restart script (will resume from checkpoint)
- Check system resources (RAM, CPU)

5. Import Error

Symptoms:

```
ModuleNotFoundError: No module named 'cloudscraper'
```

Solution:

```
pip install cloudscraper beautifulsoup4
```

6. JSON Decode Error

Symptoms:

```
JSONDecodeError: Expecting value: line 1 column 1
```

Solutions:

- Input file corrupted or invalid
- Validate JSON: `python -m json.tool cardmarket_expansion_list.json`
- Re-download input file
- Check file encoding (must be UTF-8)

Performance Optimization

Faster Execution

To speed up (at risk of more blocks):

```
PHASE1_MAX_WORKERS = 12          # More parallel workers
PHASE1_REQUESTS_PER_SECOND = 5    # Faster requests
```

More Reliable Execution

To increase success rate (slower):

```
PHASE1_MAX_WORKERS = 4          # Fewer workers
PHASE1_REQUESTS_PER_SECOND = 2    # Slower rate
PHASE1_MAX_RETRIES = 3          # More retries
```


Debugging Rejected Expansions

1. Check error details:

```
cat cardmarket_rejected_expansions.json | python -m json.tool
```

2. Inspect HTML error:

Look for `html_snippet` in errors to see what HTML was returned

3. Manual verification:

Visit expansion URL manually to confirm issue

4. Retry specific expansion:

- Remove from input JSON
- Run script
- Manually add back and re-run

Data Validation

Check card count:

```
cat cardmarket_card_list.json | grep "card_id" | wc -l
```

Check for missing fields:

```
cat cardmarket_card_list.json | python -c "  
import json, sys  
cards = json.load(sys.stdin)  
for card in cards:  
    if not card.get('card_number'):  
        print(f'Missing number: {card[\"card_name\"]} ({card[\"card_id\"]}')
```

Verify expansion coverage:

```
python -c "  
import json  
expansions = json.load(open('cardmarket_expansion_list.json'))  
empty = json.load(open('cardmarket_empty_expansions.json'))  
rejected = json.load(open('cardmarket_rejected_expansions.json'))  
cards = json.load(open('cardmarket_card_list.json'))  
  
success = len(set(c['expansion_id'] for c in cards))  
total = len(expansions)  
print(f'Success: {success}/{total} ({success/total*100:.1f}%)')  
print(f'Empty: {len(empty)}')
```

Technical Notes

Thread Safety

- File I/O uses `threading.Lock` for atomic writes
- Session pool uses locks for worker isolation
- Rate limiter uses locks for global timing
- No race conditions in multi-threaded execution

Memory Usage

- Estimated 200-500 MB RAM for full run
- Card list stored in memory until save
- Checkpoints written to disk incrementally
- Large expansions (1000+ cards) processed in chunks

Network Considerations

- Total requests: ~2500-3000 (one per expansion + pagination)
- Total data downloaded: ~500-800 MB HTML
- Average request size: 250-300 KB per page
- Estimated bandwidth: 1-2 GB total

Execution Time

Expected duration:

- Phase 1: 1.5-2.5 hours (2400 expansions, 8 workers)
- Phase 2: 15-45 minutes (50-100 rejections, careful retries)
- **Total: 2-3 hours for complete run**

Factors affecting speed:

- Network latency
- Server response time
- Rate limiting
- Number of rejections requiring Phase 2

Best Practices

Before Running

1. ✓ Verify input file is complete and valid JSON
2. ✓ Ensure stable internet connection
3. ✓ Check available disk space (500 MB free)
4. ✓ Run during off-peak hours if possible
5. ✓ Backup previous output files if re-running

During Execution

1. ✓ Monitor console output for errors
2. ✓ Don't interrupt during saves (wait for next progress line)
3. ✓ Check empty vs rejected counts (empty is normal, high rejected is problem)
4. ✓ If many 403/429 errors, stop and wait

After Completion

1. ✓ Validate output files (check file sizes)
2. ✓ Review rejected expansions (investigate if > 5%)
3. ✓ Back up output files before modifications
4. ✓ Clean up checkpoint files (automatic, but verify)

Data Maintenance

1. ✓ Re-run monthly to update card availability
2. ✓ Keep historical snapshots for price tracking
3. ✓ Merge new data with existing datasets carefully
4. ✓ Validate card IDs haven't changed (use card_id as primary key)

Conclusion

The Cardmarket Card List Scraper is a robust, production-ready tool for extracting Yu-Gi-Oh! card data from [Cardmarket.com](https://cardmarket.com). Its two-phase architecture balances speed and reliability, while comprehensive error handling and empty detection ensure accurate, complete data collection.

Key strengths:

- Intelligent empty detection (three-tier system)
- Resilient error handling (retry logic, session management)
- Anti-detection measures (rate limiting, realistic headers)

- Resumable execution (checkpoint system)
- Comprehensive data extraction (card number, rarity, metadata)

Limitations:

- Requires stable internet connection
- Subject to Cardmarket's rate limiting
- Cannot handle captcha challenges
- HTML structure changes may require updates

For questions, issues, or enhancements, refer to the conversation history or modify configuration parameters as needed.



1. <https://realpython.com/python-web-scraping-practical-introduction/>
2. https://www.reddit.com/r/web scraping/comments/19brb49/advice_on_best_method_to_scrape_codebase/
3. <https://hbs-rs.github.io/hbsgrid-docs/tutorials/PythonWebScrape/>
4. <https://guides.lib.utexas.edu/web-scrapping/scraping-with-python>
5. <https://thunderbit.com/blog/python-scraping-best-practices-tutorial>
6. <https://www.scrapingbee.com/blog/web-scraping-101-with-python/>
7. <https://scrapingant.com/blog/top-web-scraping-mistakes-python>
8. <https://stackoverflow.com/questions/21252847/what-is-the-best-practice-for-writing-maintainable-web-scrapers>